

Gestiona Gateway API

ASP.NET Core gateway for the Gestiona API. It exposes endpoints for processes, documents, thirds, activities, queue connectors, and add-on authorizations while keeping Gestiona-specific HTTP and response handling in a reusable core library.

The current application version is **1.9.0**. See [CHANGELOG.md](#) for the release history.

Requirements

- .NET 8 SDK. The repository's `global.json` selects SDK 8.0.425 or a compatible later patch.
- Access to a Gestiona API instance and the required access tokens.

Project structure

- `GestionaGatewayAPI.csproj` - ASP.NET Core API, controllers, middleware, and application startup.
- `GestionaGateway.Core/` - Gestiona client, service interfaces, implementations, configuration, and shared models.
- `GestionaGatewayAPI.Tests/` - xUnit test suite.

Configuration

The API reads Gestiona settings from the `Gestiona` configuration section:

Setting	Environment variable	Purpose
<code>Gestiona:GestionaApiBaseUrl</code>	<code>Gestiona__GestionaApiBaseUrl</code>	Base URL of the Gestiona API.
<code>Gestiona:AccessToken</code>	<code>Gestiona__AccessToken</code>	Default Gestiona access token.
<code>Gestiona:AddonToken</code>	<code>Gestiona__AddonToken</code>	Token used by add-on authorization operations.

Use environment variables for secrets and do not commit access tokens. Requests that support `X-User-Access-Token` use that header in preference to the configured default access token.

Build and run

Restore and build the solution:

```
dotnet restore GestionaGatewayAPI.sln
dotnet build GestionaGatewayAPI.sln
```

Run the API locally:

```
dotnet run --project GestionaGatewayAPI.csproj
```

The development profiles listen on `http://localhost:5123` and `https://localhost:7217`. Swagger UI is available in the Development environment at `/swagger`.

Example requests can be run from `GestionaGatewayAPI.http` in an editor with HTTP file support.

Tests

Run the complete test suite with:

```
dotnet test GestionaGatewayAPI.sln
```

Logging

The application uses Serilog for console and rolling file logging. Local log files are written under `logs/`, which is excluded from Git.

Publishing

The PowerShell publishing script builds the API in Release configuration and writes the deployable output to `C:\publish\GestionaGatewayAPI:`

```
.\publish.ps1
```

Pass `-Configuration Debug` when a Debug publish is required.